HW #4.3: Reproducible Data Analysis

Shlok Channawar

2026-04-16

Busiest Airports Analysis

Introduction

In this section we will analyze passenger traffic trends across six of the busiest airports in the world from 2020 to 2025. Our goal is to compare how different airports were affected by the COVID-19 pandemic and identify patterns in decline and recovery.

Data and Visualization

Table 1: Annual Passenger Traffic for Six Major Airports (2020-2025)

Code	2020	2021	2022	2023	2024	2025
ATL	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	106,302,208
DXB	25,836,771	29,110,609	66,069,981	86,994,365	92,331,506	95,200,000
HND	31,055,210	25,876,429	50,290,705	78,719,302	85,900,167	91,430,000
LHR	22,111,469	19,395,000	61,614,508	79,183,364	83,884,572	84,463,000
FRA	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	63,189,666
PKX	16,090,000	25,051,012	36,361,000	39,414,905	49,441,029	53,618,949

COVID-19 devastated passenger traffic in 2020 across all six airports. ATL, normally the world's busiest, dropped to just 42.9 million passengers, while LHR and FRA fell below 23 million. PKX had only opened in 2019 and recorded just 16 million in its first full pandemic year. The plot shows a steep recovery from 2021 to 2023 as travel restrictions lifted, with most airports nearly doubling traffic year over year. ATL maintained its top position throughout, peaking at 108 million in 2024. A notable pattern visible in the plot but easy to miss in the table is LHR dropping below PKX and FRA in 2021 before recovering strongly. By 2024-2025, traffic leveled off for most airports, though PKX continued growing every single year without exception and has not yet plateaued.

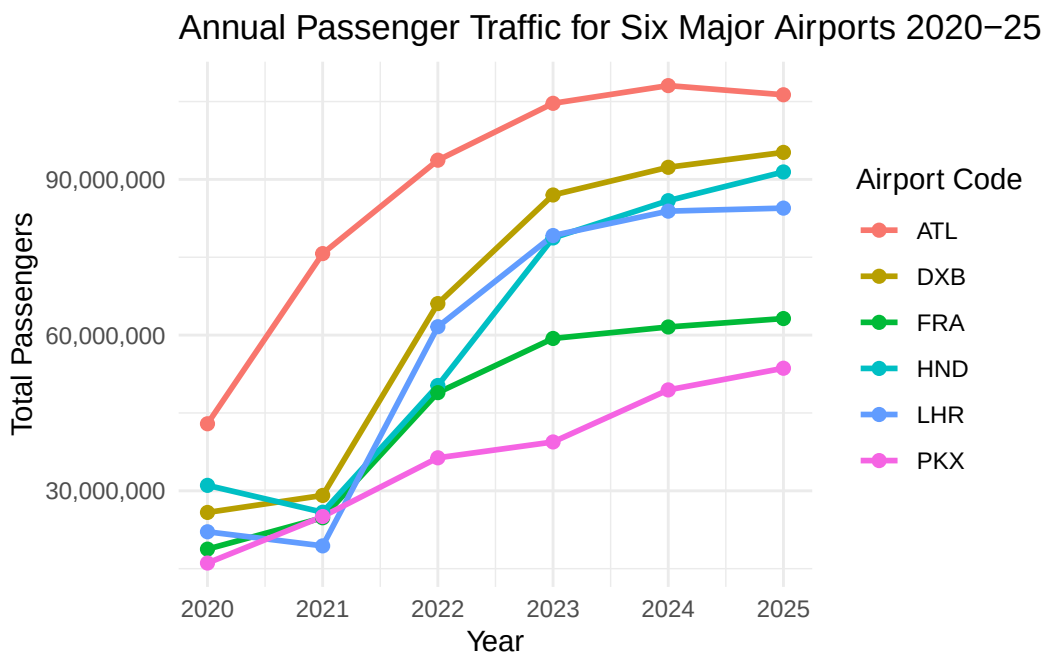


Figure 1: Annual passenger traffic for six major airports from 2020 to 2025.

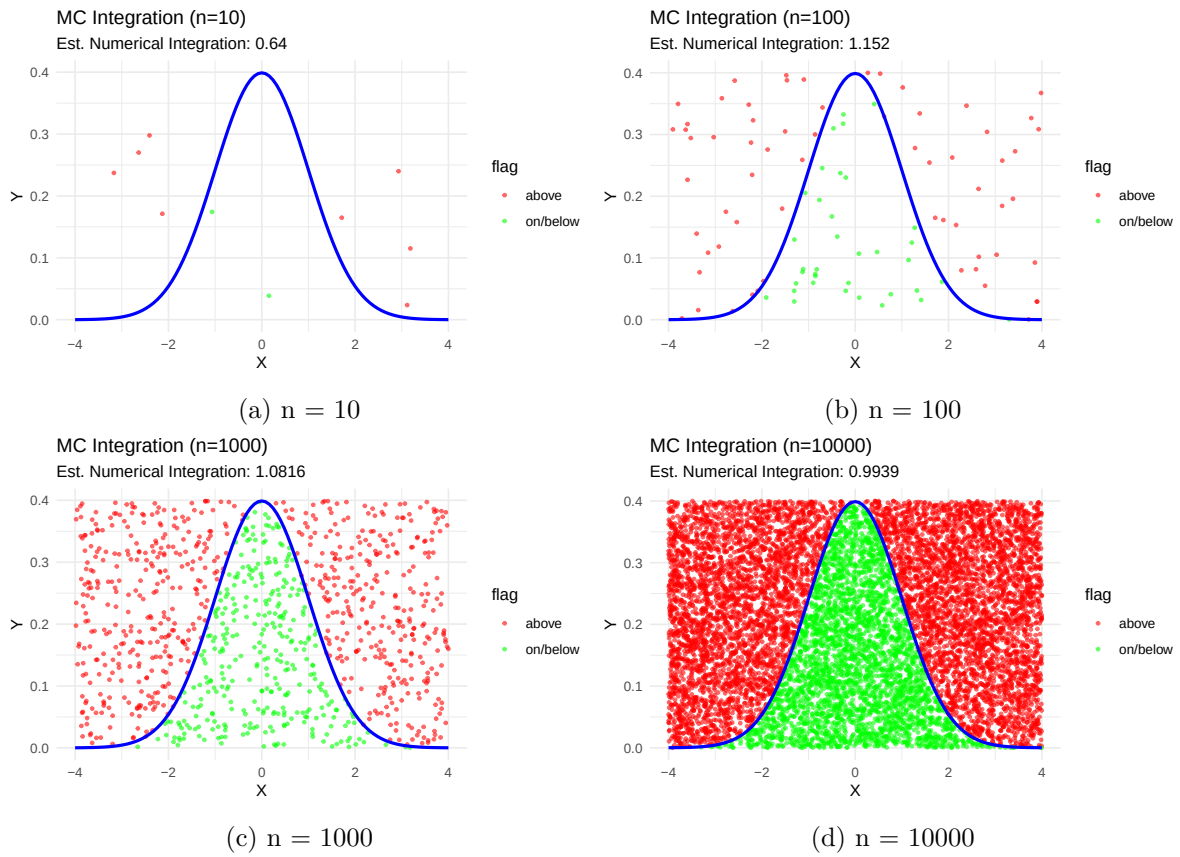
Monte Carlo Numerical Integration

Introduction

This section uses Monte Carlo numerical integration to estimate the area under the probability density function of the standard normal distribution over the interval $-4 \leq x \leq 4$, with $0 \leq y \leq 0.4$. The simulation is run at four resolutions (n = 10, 100, 1000, 10000) to show how accuracy improves with more points.

Simulation and Visualization

The small multiple makes it clear that more points produce a better estimate. At n=10 the estimate is noticeably off since there are barely any points to work with. By n=10000 the estimate converges to approximately 1.0 , very close to the true value of 1. The true answer is 1 because the standard normal distribution is a probability density function, meaning the total area under the curve must equal 1 by definition. Integrating from -4 to 4 captures nearly all of that area since values beyond 4 standard deviations are extremely rare. The n=10000 panel is the most convincing visually, with green points filling almost perfectly under the bell curve.



(a) n = 10

(b) n = 100

(c) n = 1000

(d) n = 10000

Figure 2: Small multiple showing Monte Carlo numerical integration of the standard normal distribution at four resolutions

Planning and Prompting GenAI Tools

My Plan

Goal: Tidy the calcium dataset and create a visualization that helps explore differences in ulnar calcium measurements across time and between the two treatment groups (null vs. diet/exercise).

Nouns: calcium data, treatment group, time point, ulnar calcium measurement, tidy data frame, line plot, average, individual participant lines

Verbs: read, pivot, rename, mutate, group, summarize, plot, compare

Steps:

1. Read in the calcium.csv file using `read_csv()`
2. The data is not tidy - there are 8 columns total, the first four (Initial, Year 1, Year 2, Year 3) belong to the null treatment group and the last four belong to the diet/exercise group. Pivot each set of four columns separately to long format.
3. After pivoting, add a group column to each that labels rows as either "Null" or "Diet/Exercise", then bind the two together into one data frame.
4. Clean up the time variable so the values read as "Initial", "Year 1", "Year 2", "Year 3" in the correct order using a factor.
5. Calculate the average ulnar calcium measurement at each time point for each group.
6. Create a line plot with time on the x-axis and ulnar calcium on the y-axis, showing individual participant lines faintly in the background and a bold group average line on top, colored by treatment group.

Plan-Informed Response

Using the plan above along with the calcium.csv file Claude (claude-sonnet-4-6) was prompted to write R code to tidy the data and produce a visualization. The LLM correctly identified the duplicate column name issue in the raw CSV and handled it by selecting the renamed columns (y0, y1, y2, y3 for null and y0.1, y1.1, y2.1, y3.1 for diet/exercise). It added participant IDs using row_number() before pivoting to preserve individual trajectories for plotting. The resulting plot shows individual participant lines faintly in the background with bold group average lines on top, colored by treatment group.

Generic Response

The following is the output from Anthropic Claude (claude-sonnet-4-6) when given a generic prompt with no plan: *"Create a data visualization that will help me understand these data. Calcium.csv"*

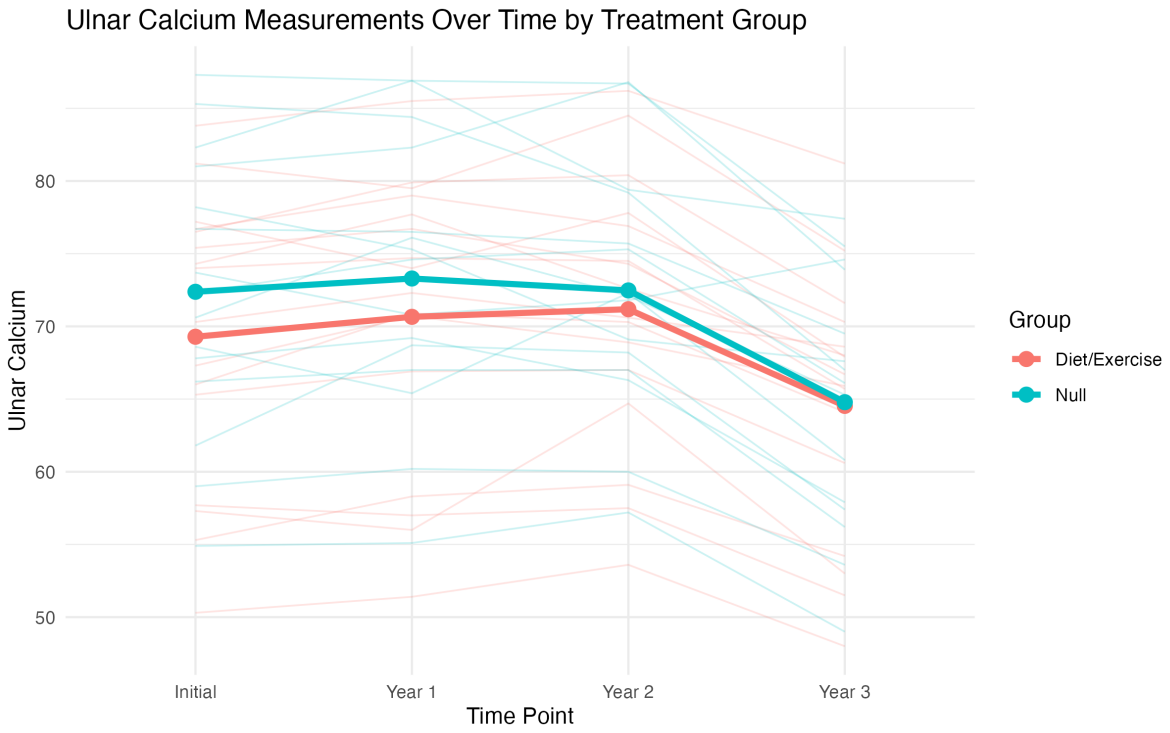


Figure 3: Ulnar calcium measurements over time by treatment group, from plan-informed Claude response.

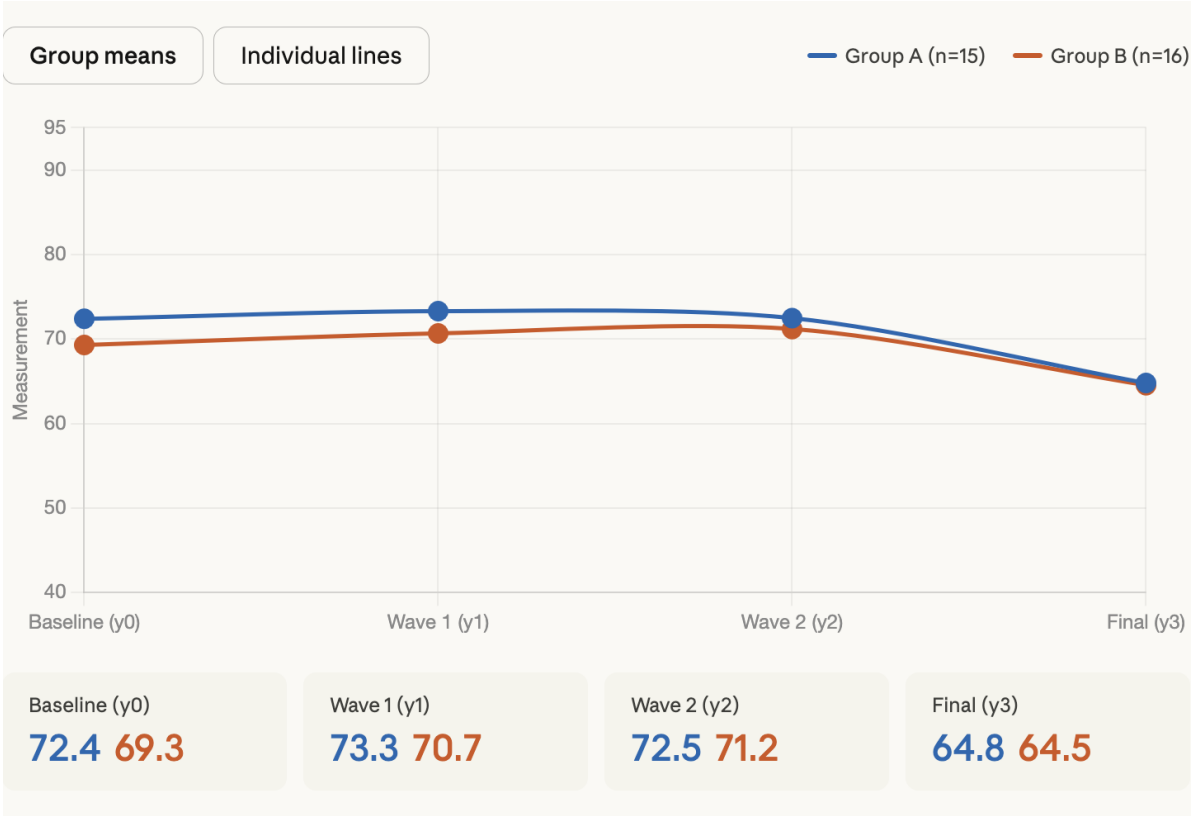


Figure 4: Generic prompt response - group means view.

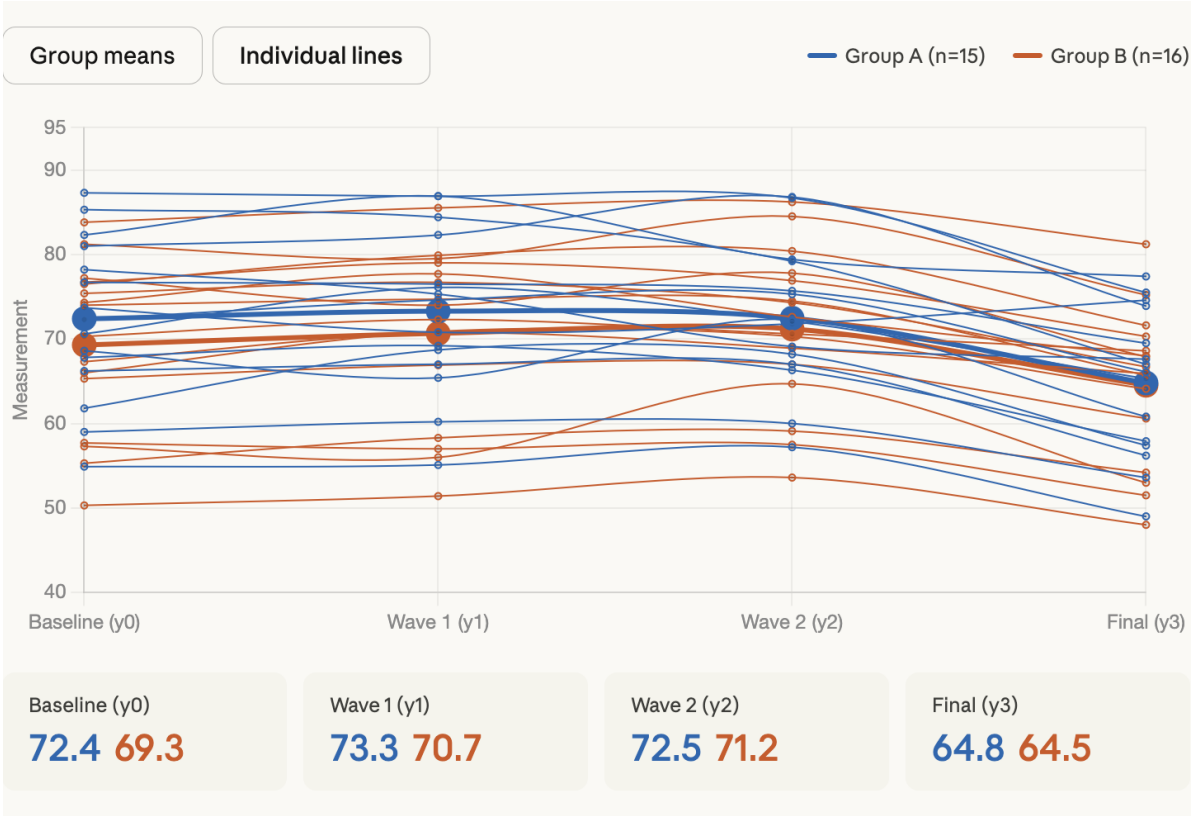


Figure 5: Generic prompt response - individual lines view.

Claude also noted that both groups follow the same pattern, staying relatively stable across the first three time points before dropping noticeably at the final measurement. Group A started slightly higher at baseline (mean 72.4 vs. 69.3) but both groups converged to nearly identical final values (64.8 and 64.5). Claude also flagged substantial individual variation and noted it could not interpret the groups without knowing what they represented.

Comparison

The plan-informed response produced cleaner amd more interpretable R code that directly addressed the context of the study. By specifying the treatment group labels (Null vs. Diet/Exercise), the correct time point names (Initial, Year 1, Year 2, Year 3) and the individual participant lines layered under group averages, the output was immediately usable and contextually accurate. The generic response, while visually polished, used generic labels like "Group A" and "Group B" and "Wave 1/Wave 2" which strip away the actual meaning of the study. It also produced an interactive widget rather than static R code, making it harder to reproduce or embed in a document. The key lesson is that a detailed plan dramatically reduces the amount of cleanup needed after the AI responds and produces output that is specific to the context rather than a generic template.

Reflection

Coming into this course with some prior R experience, the biggest shift has been moving from writing code that just works to writing code that communicates properly. The approach of writing goals, nouns, verbs, and steps before touching any code was completely new to me. In the airports analysis for example, having a clear plan upfront meant I knew exactly that I needed both a wide-format table and a long-format plot, which are two different data structures from the same dataset. Writing a function that takes n as an input and returns a full plot and then calling it four times inside a Quarto layout, felt different from just writing a script top to bottom. It forced me to think about what changes between runs and what stays the same.

The GenAI section was also surprising in terms of how much the quality of a prompt matters. The plan-informed response from Claude was immediately usable with correct group labels and time point names, while the generic one used "Group A", "Group B", and "Wave 1/Wave 2" which required interpretation just to figure out what the groups even represented. Looking back in the last 2-3 months the area I struggled with most was thinking about data structure before writing code. Early in the course I would jump straight into coding without a clear plan, which led to a lot of backtracking. If I were to redo assignments from the beginning, I would spend more time on the planning step and treat it as essential rather than optional busywork.

Appendix A: GenAI Usage

Planning and Prompting GenAI Tools Section

Tool: Anthropic Claude (claude-sonnet-4-6)
Date: April 15, 2026

Prompt 1: hey so you will help me write R code for the calcium data set ill upload with this, ill provide nouns, verbs and goals with that. you then write the code and give it to me. if this sounds good lemme know or if u have any questions before that, please state but dont just do stuff without asking me okay? **Response 1:** Sounds good! Go ahead and upload the dataset and share the nouns, verbs, and goals whenever you're ready. I'll wait before doing anything.

Prompt 2: Goal: Tidy the calcium dataset and create a visualization that helps explore differences in ulnar calcium measurements across time and between the two treatment groups (null vs. diet/exercise). Nouns: calcium data, treatment group, time point, ulnar calcium measurement, tidy data frame, line plot, average, individual participant lines Verbs: read, pivot, rename, mutate, group, summarize, plot, compare Steps: 1) Read in the calcium.csv file using read csv

2) The data is not tidy - there are 8 columns total, the first four (Initial, Year1, Year2, Year3) belong to the null treatment group and the last four belong to the diet/exercise group. Pivot each set of four columns separately to long format.

3)After pivoting, add a group column to each that labels rows as either "Nll" or "Diet/Exercise", then bind the two together into one data frame

4) Clean up the time variable so the values read as "Initial", "Year 1", "Year 2", "Year 3" in the correct order using a factor

5) calculate the average ulnar calcium measurement at each time point for each group

6)Create a line plot with time on the x-axis and ulnar calcium on the y-axis, showing individual participant lines faintly in the background and a bold group average line on top, colored by treatment group

Response 2: Few quick questions before I write anything:

1. What's the exact filename/path you're uploading? Just want to make sure the `read.csv()` call matches exactly.
2. For the individual participant lines in step 6 – does each row in the original data represent one participant, or is there a participant ID column I should know about?
3. Any preference on the plotting library – base R or `ggplot2`?

Prompt 3: my bad, I forgot to upload the file, ill upload with this prompt

also each row represents one participant, no ID column and use ggplot2

Response 3: No worries! I can see the file now. Let me take a quick look at it before writing anything.Hmm, doesn't look like the file came through. Can you try uploading it again? Sometimes it doesn't attach properly.

Prompt 4: lol my bad (file attached with this one)

Response 4: Got it. I can see the structure clearly. One thing to flag: the header row has duplicate column names (y0, y1, y2, y3, y0, y1, y2, y3) which R will auto-rename to avoid conflicts. I'll handle that in the code. Here's everything:

```
library(tidyverse)

# Step 1: Read in the data
calcium_raw <- read.csv("calcium.csv")

# Step 2: Pivot each group of 4 columns separately
# R renames duplicate headers to y0, y1, y2, y3, y0.1, y1.1, y2.1, y3.1
null_long <- calcium_raw %>%
  select(y0, y1, y2, y3) %>%
  mutate(id = row_number()) %>%
  pivot_longer(cols = -id, names_to = "time", values_to = "ulnar_calcium")

diet_long <- calcium_raw %>%
  select(y0.1, y1.1, y2.1, y3.1) %>%
  mutate(id = row_number()) %>%
  pivot_longer(cols = -id, names_to = "time", values_to = "ulnar_calcium")

# Step 3: Add group labels and bind together
null_long <- null_long %>% mutate(group = "Null")
diet_long <- diet_long %>% mutate(group = "Diet/Exercise")
calcium <- bind_rows(null_long, diet_long)

# Step 4: Clean up time variable as an ordered factor
calcium <- calcium %>%
  mutate(time = recode(time,
    "y0"   = "Initial",
    "y0.1" = "Initial",
    "y1"   = "Year 1",
    "y1.1" = "Year 1",
    "y2"   = "Year 2",
```

```r
    "y2.1" = "Year 2",
    "y3"   = "Year 3",
    "y3.1" = "Year 3"
  )) %>%
  mutate(time = factor(time, levels = c("Initial", "Year 1", "Year 2", "Year 3")))

# Step 5: Calculate group averages per time point
avg_calcium <- calcium %>%
  group_by(group, time) %>%
  summarize(avg_ulnar = mean(ulnar_calcium, na.rm = TRUE), .groups = "drop")

# Step 6: Plot
ggplot() +
  geom_line(
    data = calcium,
    aes(x = time, y = ulnar_calcium, group = interaction(id, group), color = group),
    alpha = 0.2, linewidth = 0.4
  ) +
  geom_line(
    data = avg_calcium,
    aes(x = time, y = avg_ulnar, group = group, color = group),
    linewidth = 1.4
  ) +
  geom_point(
    data = avg_calcium,
    aes(x = time, y = avg_ulnar, color = group),
    size = 3
  ) +
  labs(
    title = "Ulnar Calcium Measurements Over Time by Treatment Group",
    x = "Time Point",
    y = "Ulnar Calcium",
    color = "Group"
  ) +
  theme_minimal()
```

A couple things worth noting: Since there's no participant ID column, I create one with row_number() before pivoting so ggplot can draw individual lines correctly. The na.rm = TRUE in mean() handles that one participant in the Diet/Exercise group who only has data for some time points (row 17 in the raw file).

Appendix B: Code Appendix

```
#loads required libraries
library(tidyverse)
library(knitr)
library(scales)
airports <- tibble(
  airport = c(
    rep("Hartsfield-Jackson Atlanta International", 6),
    rep("Dubai International Airport", 6),
    rep("Tokyo Haneda Airport", 6),
    rep("Heathrow Airport", 6),
    rep("Frankfurt Airport", 6),
    rep("Beijing Daxing International Airport", 6)
  ),
  code = c(
    rep("ATL", 6), rep("DXB", 6), rep("HND", 6),
    rep("LHR", 6), rep("FRA", 6), rep("PKX", 6)
  ),
  year = as.integer(rep(2020:2025, 6)),
  passengers = c(
    42918685, 75704760, 93699630, 104653451, 108067766, 106302208,
    25836771, 29110609, 66069981, 86994365, 92331506, 95200000,
    31055210, 25876429, 50290705, 78719302, 85900167, 91430000,
    22111469, 19395000, 61614508, 79183364, 83884572, 84463000,
    18768601, 24812849, 48918482, 59355389, 61564957, 63189666,
    16090000, 25051012, 36361000, 39414905, 49441029, 53618949
  )
)
airports |>
  mutate(passengers = format(passengers, big.mark = ",")) |>
  pivot_wider(names_from = year, values_from = passengers) |>
  select(-airport) |>
  knitr::kable(
    col.names = c("Code", "2020", "2021", "2022", "2023", "2024", "2025"),
    align = "lrrrrrr",
    caption = "Annual Passenger Traffic for Six Major Airports (2020-2025)"
  )
# Line plot showing passenger trends over time coloured by airport code
ggplot(airports, aes(x = year, y = passengers, color = code, group = code)) +
  geom_line(linewidth = 1) +
```

```r
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Annual Passenger Traffic for Six Major Airports 2020-25",
    x = "Year",
    y = "Total Passengers",
    color = "Airport Code"
  ) +
  theme_minimal()
# Generate n random (x, y) coordinate pairs within specified bounds
monte_carlo_points <- function(n, x_min, x_max, y_min, y_max) {
  tibble(
    x = runif(n, min = x_min, max = x_max),
    y = runif(n, min = y_min, max = y_max)
  )
}

# run simulation at a given resolution and thren return a ggplot
make_mc_plot <- function(n) {
  sim_data <- monte_carlo_points(n = n, x_min = -4, x_max = 4, y_min = 0, y_max = 0.4) |>
    mutate(flag = ifelse(y <= dnorm(x, mean = 0, sd = 1), "on/below", "above"))

  est_integration <- mean(sim_data$flag == "on/below") * 3.2    #Estimate integral = proporti

  ggplot(sim_data, aes(x = x, y = y, color = flag)) +
    geom_point(size = 0.8, alpha = 0.6) +
    stat_function(
      fun = dnorm,
      args = list(mean = 0, sd = 1),
      xlim = c(-4, 4),
      color = "blue",
      linewidth = 1
    ) +
    scale_color_manual(values = c("above" = "red", "on/below" = "green")) +
    labs(
      title = paste0("MC Integration (n=", n, ")"),
      subtitle = paste("Est. Numerical Integration:", round(est_integration, 4)),
      x = "X",
      y = "Y",
      color = "flag"
    ) +
```

```
    theme_minimal()
}
make_mc_plot(10)
make_mc_plot(100)
make_mc_plot(1000)
make_mc_plot(10000)
knitr::include_graphics("/Users/shloki/Desktop/stat184/hw4.3/CalciumPlanPlot.png")
knitr::include_graphics("/Users/shloki/Desktop/stat184/hw4.3/ClaudePlot1.png")
knitr::include_graphics("/Users/shloki/Desktop/stat184/hw4.3/ClaudePlot2.png")
```